

OTBN SystemC TLM High-Level Design Specification

Document Version: 1.0 **Date:** 2025-11-03 **IP Module:** OTBN (OpenTitan Big Number Accelerator) **Modeling Approach:** SystemC TLM2.0 with Algorithm-Based Execution Model

Executive Summary

This document specifies the SystemC Transaction-Level Model (TLM) for the OpenTitan Big Number (OTBN) accelerator. The OTBN model implements a **novel algorithm-based execution architecture** that fundamentally differs from traditional processor modeling approaches.

Key Architectural Innovation

Unlike conventional processor models that use Instruction Set Simulators (ISS), the OTBN SystemC model:

- **Does NOT implement a processor core or ISS** - eliminates instruction fetch, decode, and execution pipeline
- **Does NOT execute firmware from IMEM** - loaded instruction memory contents are ignored
- **Invokes external C++ algorithms** - when the EXECUTE command is issued, OTBN calls algorithm implementations (e.g., RSA-2048 using OpenSSL)
- **Provides algorithm API for data access** - algorithms operate on DMEM through pointer-based interface (`execute(char* dmem)`)
- **Maintains observable behavior** - preserves register interface, state machine, interrupts, and alerts visible to software

This approach enables rapid virtual platform development while maintaining functional equivalence with RTL from a software perspective. The model focuses on the host CPU interaction model: load algorithm/data, issue EXECUTE command, wait for completion interrupt, and read results or error status.

Table of Contents

1. Introduction
2. Features
3. Configuration Parameters
4. Port Interfaces
5. Memory-Mapped Registers
6. Register Callbacks

- 7. Modeling Assumptions
 - 8. Conclusion
-

1. Introduction

1.1 OTBN Overview

The OpenTitan Big Number (OTBN) accelerator is a cryptographic coprocessor optimized for asymmetric cryptographic operations involving large integers. It provides hardware-accelerated support for operations such as RSA, Elliptic Curve Cryptography (ECC), and other public-key algorithms.

1.2 Purpose of This Document

This high-level design specification defines the SystemC TLM implementation of OTBN for virtual platform integration. It documents:

- Features included and excluded from the TLM model
- Build-time configuration parameters affecting model behavior
- External port interfaces for system-level integration
- Memory-mapped register specifications
- Functional callbacks required for register side-effects
- Key modeling assumptions and abstractions

1.3 Modeling Goals

The OTBN SystemC TLM model aims to:

1. **Enable software development** - provide sufficient functional fidelity for firmware and driver development
2. **Support system integration** - model external interfaces (EDN, Key Manager, Life Cycle Controller) at transaction level
3. **Maintain security boundaries** - implement memory protection, secure wipe operations, and alert generation
4. **Abstract implementation details** - eliminate cycle-accurate timing, ISA execution, and RTL-specific features
5. **Leverage algorithm libraries** - use established cryptographic libraries (OpenSSL) for algorithm implementations

1.4 Target Audience

This document is intended for:

- SystemC model developers implementing the OTBN TLM
- Virtual platform architects integrating OTBN into system models
- Software engineers developing firmware and drivers against the virtual platform

- Verification engineers creating test plans for the SystemC model
-

2. Features

This section classifies OTBN features into those that must be modeled in SystemC TLM (“Is”) and those that should be excluded (“Is Not”). This classification is based on the OTBN architectural decision to implement an algorithm-based execution model rather than an instruction set simulator.

2.1 Critical Architectural Context

The OTBN SystemC model differs fundamentally from the RTL implementation:

- **NO processor core or ISS implementation**
- **NO instruction execution** - IMEM contents are ignored
- **Algorithm-based execution** via C++ interface
- When EXECUTE command is issued, OTBN invokes external C++ algorithm
- Algorithm accesses DMEM and registers through an API interface
- RSA-2048 algorithm using OpenSSL as reference implementation

2.2 Is (MUST Model in SystemC TLM)

2.2.1 Command Interface and Operation Control

- EXECUTE command to trigger algorithm invocation
- SEC_WIPE_DMEN command for secure data memory wipe
- SEC_WIPE_IMEM command for secure instruction memory wipe
- Command register interface for operation initiation
- Run-to-completion execution model without host interruption capability

2.2.2 Operational State Management

- State transitions between IDLE, BUSY, and LOCKED states
- BUSY_EXECUTE state during algorithm execution
- BUSY_SEC_WIPE_DMEN and BUSY_SEC_WIPE_IMEM during memory wipe operations
- BUSY_SEC_WIPE_INT during internal state secure wipe
- LOCKED state as terminal error state requiring reset
- STATUS register reflecting current operational state

2.2.3 Interrupt Generation

- Done interrupt upon operation completion
- Interrupt signaling for state transitions from BUSY to IDLE or LOCKED
- Interrupt generation for both successful completion and error conditions

2.2.4 Memory Architecture

- **Instruction Memory (IMEM):** 8 KiB, 32-bit wide, byte-addressed starting at 0
- **Data Memory (DMEM):** 4 KiB, 256-bit wide, byte-addressed starting at 0
- Host processor access to first 3 KiB of DMEM through register interface
- Protected 1 KiB region of DMEM inaccessible to host for sensitive data
- Word-aligned 32-bit access through register interface (IMEM and DMEM windows)
- Memory access restrictions based on operational state
- Algorithm API for DMEM read/write access during execution

2.2.5 Security Features

- Secure wipe operations (DMEM, IMEM, internal state) with key rotation
- Software error severity control (fatal vs. recoverable)
- Memory isolation between host-accessible and protected DMEM regions
- LOCKED state for terminal error conditions
- Fatal and recoverable alert generation

2.2.6 External Interface Transactions

- **Entropy Distribution Network (EDN)** interfaces for RND and URND entropy supply
- **Key Manager** interface for 384-bit sideload key reception (two-share masking)
- **Life Cycle Controller** interfaces for escalation and RMA (Return Merchandise Authorization) signaling
- Transaction-level modeling of request/response protocols

2.2.7 Key Management and Random Number Generation

- Sideloaded key reception from Key Manager (384-bit, two-share masked)
- RND register providing PTG.3-compliant random numbers (256-bit, EDN-sourced)
- URND register providing local PRNG-based pseudo-random numbers (256-bit)
- Key validity status from Key Manager
- Blocking behavior when entropy not available

2.2.8 Data Integrity and Error Detection

- ECC (Error Correction Code) integrity check modeling at functional level
- Error reporting through ERR_BITS register
- Fatal and recoverable error classification
- FATAL_ALERT_CAUSE register for error diagnosis

- `ILLEGAL_BUS_ACCESS` error detection for prohibited memory access

2.2.9 Checksum and Verification

- `LOAD_CHECKSUM` register computing CRC-32-IEEE over IMEM writes
- Functional checksum computation for loaded firmware verification
- Checksum update on every 32-bit aligned IMEM write

2.2.10 Instruction Counter (Mock Implementation)

- `INSN_CNT` register reflecting estimated algorithm instruction count
- Returns count from algorithm's `get_instruction_count()` method
- Clearable when OTBN is idle

2.2.11 Alert Generation

- Recoverable alert for software errors
- Fatal alert for security violations and fatal errors
- Continuous fatal alert signaling until reset

2.3 Is Not (EXCLUDE from SystemC TLM)

2.3.1 Instruction Set Architecture and Execution

- Complete OTBN ISA instruction semantics (not modeled - architecture uses algorithm API instead)
- Instruction decoding and execution pipeline (no ISS in TLM model)
- Base instruction subset operations (LW, SW, branches, jumps)
- Big number instruction subset operations (BN.LID, BN.SID, BN.MULQACC, etc.)
- Control flow instructions execution details (LOOP, LOOPI, ECALL)
- Instruction prefetch buffer mechanics (implementation detail)
- Hardware loop stack implementation details (not relevant to algorithm model)
- Call stack implementation details (not relevant to algorithm model)

2.3.2 Register File Implementation

- 32 general-purpose registers (GPR) detailed implementation
- 32 wide data registers (WDR) detailed implementation
- Control and Status Registers (CSR) internal implementation
- Wide Special Registers (WSR) internal implementation beyond sideloaded keys
- Flag groups (FG0, FG1) with C, M, L, Z flags (algorithmic detail)
- MOD register for modular arithmetic (internal to algorithm)
- ACC register for accumulator operations (internal to algorithm)

2.3.3 Low-Level Timing and Performance

- Single-cycle instruction execution timing (not applicable to algorithm model)
- Branch/jump stall cycle details (no instruction execution)
- Memory access stall cycles (abstracted at TLM level)
- RND register blocking latency details (functional behavior modeled, not cycle-accurate)
- Instruction prefetch timing behavior (not relevant without instruction execution)

2.3.4 Data Integrity Implementation Details

- 39-bit Hsiao SECDED ECC code implementation mechanics (integrity checks abstracted)
- Register file ECC protection implementation (internal implementation)
- Memory scrambling cipher implementation details (functional wipe modeled, not scrambling algorithm)
- Scrambling key derivation and rotation mechanics (abstracted to key rotation effect)
- Address scrambling remapping algorithm (not software-visible)

2.3.5 Microarchitectural Security Features

- ISPR (Intermediate Sideload Prevention Register) detailed implementation
- Register blanking for power side-channel resistance (hardware detail)
- Duplicate counter implementations for redundancy
- Cross-counting mechanisms in loop stack
- One-hot encoding checks for control signals

2.3.6 Physical and Electrical Characteristics

- Memory SRAM physical implementation (prim_ram_1p_scr details)
- RAM configuration parameters (ram_cfg_imem, ram_cfg_dmem timing details)
- Clock domain crossing implementation details (functional behavior modeled abstractly)
- Power side-channel countermeasures implementation (register blanking hardware details)
- Physical tamper detection mechanisms (beyond functional error detection)

2.3.7 RTL and Gate-Level Features

- Glitch detection implementation on register file write enables
- One-hot encoding checks for control signals
- Duplicate counter implementations for redundancy
- Cross-counting mechanisms in loop stack

- FSM state encoding details (functional states modeled, not encoding)

2.3.8 Test and Debug Features

- DV/UVM testbench infrastructure (not part of functional model)
- Formal verification properties (verification domain)
- Static analysis tool integration (offline tools)
- Simulator-specific features (Python simulator, ISS details)
- Manufacturing test modes (not used in normal operation)

2.3.9 Unimplemented and Reserved Functionality

- Reserved register fields and values
- Future ISA extensions for lattice-based cryptography (not implemented)
- Potential KMAC hardware block interface (not yet implemented)
- Reserved error codes and alert causes

2.3.10 Integration-Specific Features

- TL-UL bus protocol implementation details (abstracted to TLM target socket)
- Comportability guideline compliance details (integration-level concern)
- OpenTitan top-level integration specifics (system-level concern)
- OTP controller scrambling key derivation details (external to OTBN functional model)

2.4 Notes

1. The SystemC TLM model focuses on the functional behavior observable by software and the algorithm execution interface.
2. The fundamental architectural difference (algorithm-based vs. ISS-based) means that instruction execution details, ISA semantics, and processor microarchitecture features are explicitly excluded.
3. Security features are modeled at the functional level (secure wipe effects, memory isolation, key sideloading) without implementing the underlying cryptographic or ECC mechanisms.
4. External interfaces (EDN, Key Manager, Life Cycle Controller) are modeled at the transaction level with appropriate abstractions for system-level integration.
5. Error detection and handling are modeled to capture software-visible behavior and system-level alerts, without implementing the detailed integrity checking mechanisms.
6. The model emphasizes the host CPU interaction model: load algorithm/data, issue EXECUTE command, wait for completion interrupt, read results or error status.

3. Configuration Parameters

This section identifies the essential build-time configuration parameters for the OTBN SystemC TLM implementation. These parameters are static capability attributes fixed at IP synthesis time or SoC integration, affecting the functional behavior of the SystemC model.

3.1 Architectural Context

The OTBN SystemC TLM model uses an **algorithm-based execution model** rather than an Instruction Set Simulator (ISS). Key architectural decisions:

- NO processor core or ISS implementation
- IMEM contents are ignored (not executed)
- External C++ algorithms access DMEM through an API
- When EXECUTE command is issued, OTBN invokes the external algorithm
- RSA-2048 algorithm implemented using OpenSSL

This architectural approach significantly affects which parameters are relevant to the TLM model.

3.2 Configuration Parameters Table

Parameter Name	Type	Default Value	Description	TLM Relevance
IMEM_SIZE	Integer	8192 bytes (8 KiB)	Instruction memory size. Defines address range for host loading operations and secure wipe target size.	Algorithm ignores IMEM contents, but host loads firmware for secure wipe and checksum computation.

Parameter Name	Type	Default Value	Description	TLM Relevance
DMEM_SIZE	Integer	4096 bytes (4 KiB)	Data memory size. Total DMEM accessible by algorithm via API. Host accesses first 3 KiB through register interface.	Core parameter for algorithm API - algorithms operate on DMEM array. Protected last 1 KiB not accessible by host.
DMEM_HOST_ACCESSIBLE_SIZE	Integer	3 KiB	Size of DMEM region accessible by host through register interface. Enforces memory protection boundary between host-accessible and algorithm-private regions.	Critical for memory isolation - algorithm can access full DMEM, host limited to first 3 KiB.
EDN_RND_WIDTH	Integer	256 bits	Width of RND register receiving entropy from EDN for PTG.3-compliant random number generation.	Affects RND register size and EDN transaction protocol (8x 32-bit transactions per RND value).
EDN_URND_WIDTH	Integer	256 bits	Width of URND seed entropy from EDN for local PRNG.	Affects URND PRNG seeding transaction size from EDN.

Parameter Name	Type	Default Value	Description	TLM Relevance
KEYMGR_KEY_WIDTH	Integer	32 bits	Width of sideload key from Key Manager (two-share masked).	Affects algorithm API key parameter size and Key Manager transaction protocol.
NUM_KEY_SHARES	Integer	2	Number of key shares for sideload key masking from Key Manager.	Defines key share structure passed to algorithm API (share0 + share1 = 384 bits each).
ECC_REG_WIDTH	Integer	32 bits	Width of registers protected by ECC (base register width).	Affects integrity check parameters in error detection logic.
ECC_REG_PARITY_WIDTH	Integer	12 bits	Parity width for 32-bit register ECC (39-bit Hsiao SECDED).	Defines integrity protection overhead; functional check abstracted in TLM.
ECC_DMEN_WIDTH	Integer	256 bits	Width of DMEM words protected by ECC.	Affects integrity check parameters for DMEM access error detection.
ECC_DMEN_PARITY_WIDTH	Integer	12 bits	Parity width for 256-bit DMEM word ECC (Hsiao SECDED).	Defines integrity protection overhead for DMEM; functional check abstracted in TLM.

Parameter Name	Type	Default Value	Description	TLM Relevance
SCRAMBLE_KEY_WIDTH	reg	256 bits	Width of PRESENT cipher key for memory scrambling.	Affects secure wipe key rotation modeling (functional effect, not cipher implementation).
SCRAMBLE_NONCE_WIDTH	reg	128 bits	Width of nonce for memory scrambling diversification.	Affects secure wipe diversification modeling (functional effect, not cipher implementation).

3.3 Parameters Explicitly Excluded

The following are **NOT** included as they fail one or more criteria:

3.3.1 Runtime-Configurable Features

- **software_errs_fatal**: Configured at runtime via CTRL register, not a build-time parameter
- **Memory access permissions during BUSY state**: Enforced by operational state machine, not a build-time configuration

3.3.2 Not Relevant to Algorithm-Based Model

- **Number of GPRs/WDRs**: Register file details not exposed in algorithm API model
- **Instruction execution timing**: No ISS implementation, no instruction-level timing
- **Hardware loop stack depth**: Control flow managed by C++ algorithm, not instruction execution
- **Call stack depth**: Not relevant without instruction execution

3.3.3 Integration-Only Attributes

- **Number of OTBN instances**: System integration attribute, not affecting single IP functional behavior
- **Base address/memory mapping**: TLM target socket abstracts address decoding

- **Clock domain assignments:** TLM abstraction level does not model clock domains functionally
- **TL-UL bus parameters:** Abstracted to TLM target socket interface

3.3.4 Physical/Implementation Details

- **ECC polynomial specifics:** Functional model checks integrity violations, not ECC encoding details
- **Scrambling cipher details:** Functional model implements secure wipe effect, not cipher mechanics
- **SRAM physical configuration:** Physical implementation below TLM abstraction level

3.4 Notes

1. **Algorithm-Centric Design:** The configuration focuses on parameters that affect the C++ algorithm API, memory sizes accessible through the API, and external interfaces (EDN, Key Manager).
2. **Memory Architecture:** Both IMEM and DMEM parameters are included despite IMEM not being executed, as the model must support host loading operations and secure wipe commands.
3. **Security Features:** Parameters related to key sideloading, memory protection boundaries, and scrambling key widths are essential for modeling security boundaries and secure wipe functionality.
4. **Data Integrity:** ECC parameters define the integrity protection scheme used throughout the model, affecting error detection behavior observable to software.
5. **External Interfaces:** EDN and Key Manager interface parameters are included as they affect functional behavior (RND blocking, key validity checks) visible to algorithms.
6. **Exclusion Rationale:** Parameters related to instruction execution, microarchitecture, physical implementation, and runtime register configuration are excluded per the architectural decision to use an algorithm-based model.

4. Port Interfaces

This section specifies the transaction-level port interfaces for the OTBN SystemC model. The interfaces are designed for virtual platform integration using abstract TLM ports rather than signal-level RTL connectivity.

4.1 Critical Architectural Context

The OTBN SystemC model uses an algorithm-based execution approach:

- No processor core or ISS implementation
- External C++ algorithms access DMEM through an API
- TLM simple target socket for register access (NOT TL-UL signals)
- Transaction-level abstraction for all external interfaces

4.2 Port Interfaces Table

Interface Category	Port Name	Port Type	Description
Register Bus	reg_bus	tlm_target_socket	Register-mapped interface for IMEM/DMEM access, command control, status monitoring, and configuration registers
Interrupt	intr_done	sc_out	Interrupt output signaling operation completion (EXECUTE, SEC_WIPE_DMEM, SEC_WIPE_IMEM) and state transitions to IDLE or LOCKED
Alert	alert_fatal	sc_out	Fatal alert output for security violations and fatal errors, continuously asserted until reset
Alert	alert_recov	sc_out	Recoverable alert output for software errors, sent once when processor stops

Interface Category	Port Name	Port Type	Description
Protocol Transaction	edn_rnd_req	sc_port	Request interface to Entropy Distribution Network for AIS31-compliant PTG.3 random numbers used by RND register
Protocol Transaction	edn_rnd_rsp	sc_export	Response interface from Entropy Distribution Network providing 32-bit entropy transactions (8 transactions per 256-bit RND value)
Protocol Transaction	edn_urnd_req	sc_port	Request interface to Entropy Distribution Network for URND local PRNG seeding
Protocol Transaction	edn_urnd_rsp	sc_export	Response interface from Entropy Distribution Network for URND PRNG seed entropy
Protocol Transaction	keymgr_key_req	sc_port	Request interface to Key Manager for 384-bit sideload key reception

Interface Category	Port Name	Port Type	Description
Protocol Transaction	keymgr_key_rspsc_export		Response interface from Key Manager providing key validity status and two-share masked key data
Protocol Transaction	lc_escalate_req	sc_port	Request interface to Life Cycle Controller for escalation signal reception
Protocol Transaction	lc_escalate_rsp	sc_export	Response interface acknowledging Life Cycle Controller escalation requests
Protocol Transaction	lc_rma_req	sc_port	Request interface to Life Cycle Controller for RMA entry request reception
Protocol Transaction	lc_rma_rsp	sc_export	Response interface to Life Cycle Controller acknowledging RMA secure wipe completion
Clock	clk_core	sc_in	Primary clock input for OTBN core operation (abstract frequency representation)
Clock	clk_edn	sc_in	Clock input for EDN interface timing (abstract frequency representation)

Interface Category	Port Name	Port Type	Description
Reset	rst_n	sc_in	Active-low asynchronous reset input

4.3 Interface Descriptions

4.3.1 Register Bus Interface The `reg_bus` TLM target socket provides access to all OTBN registers, including:

- IMEM window (0x4000-0x5FFC): 8 KiB instruction memory access
- DMEM window (0x8000-0x8BFC): First 3 KiB of data memory access
- Control registers (CMD, CTRL, STATUS, etc.)
- Interrupt registers (INTR_STATE, INTR_ENABLE, INTR_TEST)
- Alert test registers (ALERT_TEST)
- Status and error registers (ERR_BITS, FATAL_ALERT_CAUSE, INSN_CNT, LOAD_CHECKSUM)

4.3.2 Interrupt and Alert Interfaces

- **intr_done**: Asserted when operation completes (transition from BUSY to IDLE or LOCKED). Controlled by INTR_STATE and INTR_ENABLE registers.
- **alert_fatal**: Continuously asserted for fatal errors (security violations, illegal bus access) until reset.
- **alert_recov**: Pulsed once for recoverable software errors when processor stops.

4.3.3 External Protocol Interfaces All external interfaces use abstract protocol transaction ports:

- **EDN interfaces**: Request/response for entropy delivery (RND, URND)
- **Key Manager interfaces**: Request/response for sideload key reception
- **Life Cycle Controller interfaces**: Escalation and RMA signaling

4.3.4 Clock and Reset Interfaces

- **clk_core**: Abstract frequency value (double) for core operation timing
- **clk_edn**: Abstract frequency value for EDN interface timing
- **rst_n**: Active-low reset, returns OTBN to IDLE state and clears error conditions

4.4 Notes

1. **TLM Abstraction**: All interfaces are transaction-level, not signal-level. No TL-UL bus protocol implementation.

2. **Algorithm API Not a Port:** The C++ algorithm interface is an internal API call, not an external port.
 3. **Memory Protection:** DMEM window exposes only first 3 KiB to host; protected 1 KiB accessed only by algorithm via API.
 4. **State Dependencies:** Many interfaces have access restrictions based on STATUS register value (IDLE, BUSY, LOCKED).
-

5. Memory-Mapped Registers

The OTBN module's registers are accessed through the peripheral bus at the module's base address. All configuration, status, and data transfer is handled via these registers.

5.1 Interrupt and Alert Registers

Register Name	Offset	Size (bits)	Access	Reset Value	Description/Notes
INTR_STATE	0x0	32	RW1C	0x0	Interrupt state register. Bit 0 (done) indicates OTBN has completed the operation. Write 1 to clear.
INTR_ENABLE	0x4	32	RW	0x0	Interrupt enable register. Bit 0 (done) enables interrupt when INTR_STATE.done is set.
INTR_TEST	0x8	32	WO	0x0	Interrupt test register. Write 1 to bit 0 (done) to force INTR_STATE.done to 1 for testing.

Register Name	Offset	Size (bits)	Access	Reset Value	Description/Notes
ALERT_TEST	0x0C	32	WO	0x0	Alert test register. Bit 0 (fatal) and bit 1 (recov) trigger respective alert events when written with 1.

5.2 Command and Control Registers

Register Name	Offset	Size (bits)	Access	Reset Value	Description/Notes
CMD	0x10	32	WO	0x0	Command register to initiate OTBN operations. Bits 7:0 accept commands: 0xD8=EXECUTE (start algorithm), 0xC3=SEC_WIPE_DMEM, 0x1E=SEC_WIPE_IMEM. Writes ignored if not idle.
CTRL	0x14	32	RW	0x0	Control register. Bit 0 (software_errs_fatal) configures whether software errors produce fatal errors (1) or recoverable errors (0). Writes ignored if not idle.

5.3 Status and Error Registers

Register Name	Offset	Size (bits)	Access	Reset Value	Description/Notes
STATUS	0x18	32	RO	0x0	Status register indicating current operational state. Values: 0x00=IDLE, 0x01=BUSY_EXECUTE, 0x02=BUSY_SEC_WIPE_DMEM, 0x03=BUSY_SEC_WIPE_IMEM, 0x04=BUSY_SEC_WIPE_INT, 0xFF=LOCKED. Read from internal state machine, not static storage.
ERR_BITS	0x1C	32	RW	0x0	Error classification register. Reflects detected errors during operation. Write any value to clear when OTBN idle or locked. Writes ignored if OTBN busy. Individual error bits indicate error types.

Register Name	Offset	Size (bits)	Access	Reset Value	Description/Notes
FATAL_ALERT_CAUSE	0x0	16	RO	0x0	Fatal alert cause register. Indicates reason for fatal alert. Bit 0=BAD_DATA_ADDR, Bit 1=BAD_INSN_ADDR, Bit 2=CALL_STACK, Bit 3=ILLEGAL_INSN, Bit 4=LOOP, Bit 5=KEY_INVALID, Bit 6=RND_REP_CHK_FAIL, Bit 7=RND_FIPS_CHK_FAIL, Bit 8=IMEM_INTG_VIOLATION, Bit 9=DMEM_INTG_VIOLATION, Bit 10=REG_INTG_VIOLATION, Bit 11=BUS_INTG_VIOLATION, Bit 12=ILLEGAL_BUS_ACCESS, Bit 13=LIFECYCLE_ESCALATION, Bit 14=FATAL_SOFTWARE. Persists until reset.

Register Name	Offset	Size (bits)	Access	Reset Value	Description/Notes
INSN_CNT	0x24	32	RW	0x0	Instruction count register. Returns estimated instruction count from algorithm's <code>get_instruction_count()</code> method. Write any value to clear when OTBN idle or locked. Writes ignored if OTBN busy.

5.4 Load Integrity Register

Register Name	Offset	Size (bits)	Access	Reset Value	Description/Notes
LOAD_CHECKSUM	0x28	32	RO	0x0	CRC-32-IEEE checksum computed over IMEM writes. Updated automatically on every 32-bit aligned IMEM write. Used for firmware integrity verification. Read-only, cleared on reset.

5.5 Memory Access Window Registers

Register Name	Offset	Size (bits)	Access	Reset Value	Description/Notes
IMEM	0x4000- 0x5FFC	32	RW	0x0	Instruction memory access window. 8 KiB (2048 words). Host loads algorithm firmware here. Only accessible when OTBN idle. If busy or locked, reads return 0, writes ignored. If busy, access triggers ILLEGAL- GAL_BUS_ACCESS fatal error. Word-aligned 32-bit access only, byte writes not supported.

Register Name	Offset	Size (bits)	Access	Reset Value	Description/Notes
DMEM	0x8000-0x8BFC	32	RW	0x0	Data memory access window. First 3 KiB of 4 KiB DMEM (768 words) visible to host. Used for algorithm input/output data. Only accessible when OTBN idle. If busy or locked, reads return 0, writes ignored. If busy, access triggers ILLEGAL_BUS_ACCESS fatal error. Word-aligned 32-bit access only, byte writes not supported. Remaining 1 KiB protected from host access for sensitive algorithm data.

Legend: RO = Read Only, RW = Read/Write, WO = Write Only, RW1C = Write-1-to-Clear

5.6 Notes

1. **State-Dependent Access:** Many registers have access restrictions based on STATUS register value (IDLE, BUSY, LOCKED).
2. **IMEM/DMEM Windows:** Provide 32-bit word-aligned access to memories. Protected by state machine to prevent access during operation.
3. **Error Registers:** ERR_BITS and FATAL_ALERT_CAUSE provide software-visible error diagnosis. FATAL_ALERT_CAUSE persists until reset.

4. **Command Encoding:** CMD register uses specific byte values (0xD8, 0xC3, 0x1E) to prevent accidental operation initiation.
 5. **Memory Protection:** DMEM window exposes only first 3 KiB (768 words) to host; protected 1 KiB reserved for algorithm-internal sensitive data.
-

6. Register Callbacks

This section identifies registers that require read or write callbacks in the SystemC TLM implementation. Callbacks are necessary for registers with immediate functional side-effects or conditional access dependencies.

6.1 Overview

The OTBN SystemC model uses an **algorithm-based execution model** where external C++ algorithms (e.g., RSA-2048) are invoked via an API when the EXECUTE command is written to the CMD register. This architectural approach influences which registers require callbacks for proper functional modeling.

6.2 Register Callbacks Table

Callback Name	Type	Description
handle_write_CMD	Write	Immediate Side-Effects: Triggers algorithm invocation (EXECUTE=0xD8), secure DMEM wipe (SEC_WIPE_DMEM=0xC3), or secure IMEM wipe (SEC_WIPE_IMEM=0x1E). Changes STATUS to BUSY_EXECUTE, BUSY_SEC_WIPE_DMEM, or BUSY_SEC_WIPE_IMEM. Invokes external C++ algorithm via API for EXECUTE command. Conditional Access: Writes ignored if STATUS is not IDLE (checked against current state machine state).

Callback Name	Type	Description
handle__write_CTRL	Write	Conditional Access: Writes to software_errs_fatal bit ignored if STATUS is not IDLE. Checks current state machine state before allowing write to configure error severity behavior.
handle__read_STATUS	Read	Immediate Side-Effects: Returns dynamically computed operational state from internal state machine (IDLE=0x00, BUSY_EXECUTE=0x01, BUSY_SEC_WIPE_DMEM=0x02, BUSY_SEC_WIPE_IMEM=0x03, BUSY_SEC_WIPE_INT=0x04, LOCKED=0xFF), not static register storage.
handle__write_INTR_STATE	Write	Immediate Side-Effects: Write-one-to-clear (W1C) behavior for done interrupt flag. Writing 1 to done bit clears INTR_STATE.done and de-asserts done interrupt output port if INTR_ENABLE.done is set.
handle__write_INTR_TEST	Write	Immediate Side-Effects: Writing 1 to done bit forces INTR_STATE.done to 1 for testing. If INTR_ENABLE.done is set, asserts done interrupt output port.
handle__write_ALERT_TEST	Write	Immediate Side-Effects: Writing 1 to fatal bit (bit 0) triggers fatal_alert output port. Writing 1 to recov bit (bit 1) triggers recoverable_alert output port. Used for alert testing.

Callback Name	Type	Description
handle_write_ERR_BITS	Write	Conditional Access: Writes clear register to 0 only when OTBN is not running (STATUS is IDLE or LOCKED). Write attempts while OTBN is running (any BUSY state) are ignored. Checks STATUS before allowing clear operation.
handle_read_ERR_BITS	Read	Immediate Side-Effects: Returns error classification bits set during operation errors or from internal error detection logic, reflecting current error state not static storage.
handle_write_INSN_CNT	Write	Conditional Access: Writes clear register to 0 only when OTBN is not running (STATUS is IDLE or LOCKED). Write attempts while OTBN is running (any BUSY state) are ignored. Checks STATUS before allowing clear operation.
handle_read_INSN_CNT	Read	Immediate Side-Effects: Returns mock instruction count from algorithm's <code>get_instruction_count()</code> method, not static register storage. Provides estimated instruction count for software visibility.
handle_read_FATAL_ALERT_CAUSE	Read	Immediate Side-Effects: Returns fatal alert cause from internal error detection logic, reflecting which fatal error condition occurred. Persists until reset.

Callback Name	Type	Description
handle_read_LOAD_CHECKSUM	Read	Immediate Side-Effects: Returns dynamically computed CRC-32-IEEE checksum over all IMEM writes since reset. Value updated on every IMEM write, not stored in register.
handle_write_IMEM	Write	Conditional Access: Writes only allowed when STATUS is IDLE. If STATUS is BUSY, write triggers ILLEGAL_BUS_ACCESS fatal error and moves to LOCKED state. If STATUS is LOCKED, write ignored (returns 0 on read). Immediate Side-Effects: Every 32-bit aligned write updates LOAD_CHECKSUM CRC-32-IEEE value.
handle_read_IMEM	Read	Conditional Access: Reads only allowed when STATUS is IDLE. If STATUS is BUSY or LOCKED, reads return 0. If STATUS is BUSY, read triggers ILLEGAL_BUS_ACCESS fatal error and moves to LOCKED state.
handle_write_DMEM	Write	Conditional Access: Writes only allowed when STATUS is IDLE. If STATUS is BUSY, write triggers ILLEGAL_BUS_ACCESS fatal error and moves to LOCKED state. If STATUS is LOCKED, write ignored (returns 0 on read).

Callback Name	Type	Description
handle_read_DMEM	Read	Conditional Access: Reads only allowed when STATUS is IDLE. If STATUS is BUSY or LOCKED, reads return 0. If STATUS is BUSY, read triggers ILLEGAL_BUS_ACCESS fatal error and moves to LOCKED state.

6.3 Registers Without Callbacks

The following registers do **not** require callbacks as they are pure storage registers with no immediate functional side-effects and no conditional access dependencies:

- **INTR_ENABLE:** Simple read/write storage register that enables/disables done interrupt. No side-effects on read or write, used only as a mask for interrupt generation logic.

6.4 Implementation Notes

6.4.1 Algorithm-Based Execution Model

- **CMD.EXECUTE:** When written, invokes external C++ algorithm via `otbn_algorithm::execute(char *dmem)` API. Algorithm accesses DMEM through pointer, not register writes.
- **INSN_CNT:** Returns mock instruction count from `otbn_algorithm::get_instruction_count()` method, not actual instruction execution tracking.
- **IMEM:** Contents loaded by host but ignored by model (no instruction fetch/decode). Algorithm implementation uses OpenSSL library, not OTBN ISA.

6.4.2 State Machine Dependencies Many register callbacks depend on the current operational state (STATUS):

- **IDLE (0x00):** Allows CMD writes, CTRL writes, ERR_BITS/INSN_CNT clears, IMEM/DMEM access
- **BUSY states (0x01-0x04):** Blocks most writes (CMD, CTRL, ERR_BITS, INSN_CNT clears, IMEM/DMEM); IMEM/DMEM access triggers fatal error
- **LOCKED (0xFF):** Blocks CMD writes, IMEM/DMEM access; allows ERR_BITS/INSN_CNT clears but not effective operations

6.4.3 Abstract Port Interactions Callbacks interact with abstract TLM ports:

- **done:** Interrupt output (sc_out) - triggered by INTR_STATE/INTR_TEST writes when INTR_ENABLE.done set
- **fatal_alert:** Alert output (sc_out) - triggered by ALERT_TEST writes or ILLEGAL_BUS_ACCESS/fatal errors
- **recoverable_alert:** Alert output (sc_out) - triggered by ALERT_TEST writes or recoverable errors

6.4.4 Memory Access Integrity IMEM and DMEM callbacks implement:

1. **Conditional Access Control:** Check STATUS, reject if BUSY/LOCKED
2. **Error Injection:** Trigger ILLEGAL_BUS_ACCESS fatal error if accessed while BUSY
3. **Checksum Update:** Update LOAD_CHECKSUM CRC-32-IEEE on every 32-bit aligned write
4. **State Transitions:** Move to LOCKED state on fatal errors

6.4.5 Error Handling Error-related callbacks (ERR_BITS, FATAL_ALERT_CAUSE) reflect error detection logic:

- **ERR_BITS:** Set by error detection logic during operations; clearable by host when idle
- **FATAL_ALERT_CAUSE:** Set on fatal errors, persists until reset, drives LOCKED state

6.5 Summary

This callback table comprehensively captures registers requiring functional side-effect handling in the OTBN SystemC TLM model. The callbacks are organized into two categories:

1. **Immediate Functional Side-Effects:** Registers that trigger state changes (CMD), invoke algorithms (CMD.EXECUTE), update outputs (INTR_STATE, INTR_TEST, ALERT_TEST), compute dynamic values (STATUS, INSN_CNT, LOAD_CHECKSUM), trigger errors (IMEM/DMEM when busy), or reflect live hardware state (ERR_BITS, FATAL_ALERT_CAUSE).
2. **Conditional Access Dependencies:** Registers with access restrictions based on operational state (CMD, CTRL, ERR_BITS, INSN_CNT writable only when idle; IMEM/DMEM accessible only when idle).

Pure storage registers (INTR_ENABLE) without side-effects or access dependencies are explicitly excluded from the callback list.

7. Modeling Assumptions

This section defines the key modeling assumptions for the OTBN SystemC TLM implementation. These assumptions guide the abstraction level and scope of the virtual platform model.

7.1 Algorithm Execution Model

1. **Algorithm-based execution, not instruction execution** - OTBN invokes external C++ algorithms (e.g., RSA-2048 via OpenSSL) when EXECUTE command is written; no processor core or ISS implemented.
2. **IMEM contents ignored** - Instruction memory loaded by host but not executed; firmware serves only as secure wipe target.
3. **Algorithm API for DMEM access** - C++ algorithms access DMEM through pointer-based API (`execute(char* dmem)`), not through instruction semantics.
4. **Mock instruction counter** - `INSN_CNT` returns estimated count from algorithm's `get_instruction_count()` method, not actual instruction execution count.
5. **Algorithm interface contract** - Each algorithm implements `execute()`, `get_instruction_count()`, `reset()`, and `message_objects()` methods per base class.

7.2 Memory Modeling

6. **Harvard architecture abstraction** - 8 KiB IMEM and 4 KiB DMEM modeled as byte-addressable memories; access width differences abstracted at transaction level.
7. **Protected DMEM region** - Host accesses only first 3 KiB via register interface; last 1 KiB reserved for algorithm-internal sensitive data.
8. **State-conditional memory access** - Register-based memory access blocked when BUSY (returns zero on read, ignores writes, generates `ILLEGAL_BUS_ACCESS` error).
9. **Scrambling key rotation for secure wipe** - Memory scrambling modeled functionally as key replacement effect; cipher implementation details excluded.
10. **IMEM bypass for algorithms** - Algorithm never reads IMEM; operates solely on DMEM state and KEY registers.

7.3 Timing Abstraction

11. **Approximate timing, not cycle-accurate** - Model timing abstracts operation durations to approximately match RTL observable delays without

cycle-level precision.

12. **No stall cycle modeling** - Memory access stalls, instruction fetch delays, and pipeline stalls not modeled; operations complete in abstracted time intervals.
13. **Blocking entropy requests** - RND register access blocks until EDN delivers entropy; blocking duration approximated, not cycle-accurate.
14. **Secure wipe duration estimates** - SEC_WIPE_DMED/IMEM operations model approximate wipe timing based on memory size and scrambling key rotation overhead.
15. **Algorithm execution timing** - Each algorithm provides estimated execution time through `get_execution_time()` method based on operation complexity.

7.4 External Interfaces

16. **Transaction-level external interfaces** - EDN, Key Manager, and Life Cycle Controller interfaces modeled as abstract transaction protocols, not signal-level.
17. **EDN entropy delivery abstraction** - Entropy requests/responses use simplified protocol; AIS31 PTG.3 compliance checked functionally, not at entropy source level.
18. **Key Manager sideload abstraction** - Sideload key delivered as two-share masked data structure; masking scheme functionally correct but not cryptographically implemented.
19. **Life Cycle escalation effect** - Escalation signal triggers immediate transition to LOCKED state; implementation details of life cycle FSM excluded.
20. **RMA secure wipe completion** - RMA request triggers comprehensive secure wipe (DMEM, IMEM, internal state); completion signaled to Life Cycle Controller via response interface.

7.5 Security and Error Detection

21. **Functional integrity checks, not ECC implementation** - ECC errors modeled by injecting error conditions; 39-bit Hsiao SECDED encoding/decoding not implemented.
22. **Error classification abstraction** - ERR_BITS and FATAL_ALERT_CAUSE reflect error types; internal error detection mechanisms abstracted.
23. **Alert generation timing** - Fatal alerts asserted immediately on error detection; recoverable alerts pulsed once; detailed timing abstracted.

- 24. **LOCKED state terminal condition** - LOCKED state requires full reset; no recovery mechanism modeled (matches RTL behavior).
- 25. **Checksum functional correctness** - LOAD_CHECKSUM implements CRC-32-IEEE correctly for firmware integrity; hardware optimization details excluded.

7.6 Cryptographic Operations

- 26. **OpenSSL for RSA-2048** - RSA-2048 algorithm uses OpenSSL library functions; OTBN ISA semantics not implemented.
- 27. **Algorithm output equivalence** - Algorithm produces cryptographically correct results in DMEM; internal register state (GPRs, WDRs, flags) not modeled.
- 28. **Key sideload functional usage** - Algorithms receive sideloaded key as parameter; key masking handled abstractly, not cryptographically.
- 29. **RND/URND functional correctness** - Random number generation provides statistically appropriate entropy; PTG.3 compliance abstracted to entropy source interface.
- 30. **Modular arithmetic abstraction** - Algorithms implement modular operations using library functions; MOD register and hardware modular multiplier not modeled.

7.7 State Machine and Control Flow

- 31. **State machine observable behavior** - STATUS register reflects correct state transitions; internal FSM encoding details excluded.
- 32. **Command decoding functional** - CMD register recognizes command values (0xD8, 0xC3, 0x1E); encoding rationale (Hamming distance) not modeled.
- 33. **Error-driven state transitions** - Fatal errors transition to LOCKED; recoverable errors remain in IDLE after completion; matches RTL behavior.
- 34. **Interrupt and alert protocol** - Interrupts and alerts follow OpenTitan comportability guidelines; timing details abstracted.
- 35. **Reset behavior comprehensive** - Reset clears all state, returns to IDLE, de-asserts alerts/interrupts, clears error registers; matches RTL reset behavior.

7.8 Scope Limitations

- 36. **No instruction-level debugging** - No support for instruction tracing, breakpoints, or single-step execution (not applicable to algorithm model).

37. **No power modeling** - Power consumption, clock gating, and power side-channel countermeasures not modeled.
38. **No physical fault injection** - Glitch detection, duplicate counter checks, and one-hot encoding checks not implemented.
39. **No DV/UVM infrastructure** - Model does not include UVM components, scoreboards, or coverage collection (testing uses native SystemC testbenches).
40. **No RTL implementation details** - Register blanking, cross-counting, loop stack implementation, and other RTL microarchitecture excluded.

7.9 Summary

These assumptions define the OTBN SystemC TLM model as a **functional virtual platform** suitable for:

- Software development and driver validation
- System-level integration and protocol verification
- Performance estimation (approximate timing)
- Security boundary validation (memory protection, secure wipe, key isolation)

The model explicitly **excludes**:

- Cycle-accurate timing
- Instruction set simulation
- Physical implementation details
- RTL-specific security mechanisms
- Power and side-channel analysis

This abstraction level balances simulation performance with functional fidelity appropriate for early software development and virtual platform integration.

8. Conclusion

8.1 Summary

This high-level design specification has defined the SystemC TLM implementation for the OTBN (OpenTitan Big Number accelerator) IP module. The specification emphasizes the **algorithm-based execution model**, a novel architectural approach that enables efficient virtual platform modeling without implementing a full instruction set simulator.

Key aspects of the OTBN SystemC TLM model:

1. **Algorithm-Based Architecture:** External C++ algorithms (e.g., RSA-2048 via OpenSSL) invoked through API when EXECUTE command is issued; no processor core or ISS implementation required.

2. **Memory Architecture:** 8 KiB IMEM (loaded but not executed) and 4 KiB DMEM (accessible to algorithms via pointer-based API) with memory protection enforcing 3 KiB host-accessible region.
3. **State Machine:** Accurate modeling of IDLE, BUSY, and LOCKED states with proper state transitions, interrupt generation, and alert signaling.
4. **External Interfaces:** Transaction-level modeling of EDN (entropy), Key Manager (sideload keys), and Life Cycle Controller (escalation/RMA) interfaces.
5. **Security Features:** Secure wipe operations, memory isolation, error detection and classification, and alert generation modeled at functional level.
6. **Register Interface:** Complete memory-mapped register set with appropriate callbacks for side-effects, conditional access, and dynamic value computation.

8.2 Design Principles

The OTBN SystemC TLM design adheres to these principles:

- **Functional Fidelity:** Preserve software-observable behavior while abstracting implementation details
- **Simulation Performance:** Eliminate cycle-accurate timing and instruction-level simulation for faster execution
- **Algorithm Modularity:** Support multiple cryptographic algorithms through well-defined base class interface
- **Security Boundaries:** Maintain memory protection, secure wipe guarantees, and error isolation
- **Integration Simplicity:** Provide clean TLM interfaces for system-level virtual platform integration

8.3 Implementation Roadmap

Implementation of the OTBN SystemC TLM model should proceed in phases:

Phase 1: Core Infrastructure - Register map and callback framework - State machine implementation - Memory models (IMEM, DMEM) with protection logic - Basic command interface (CMD, STATUS)

Phase 2: Algorithm Framework - Algorithm base class definition - DMEM access API implementation - Algorithm invocation mechanism - Mock instruction counter and timing

Phase 3: External Interfaces - EDN interfaces for RND/URND - Key Manager sideload interface - Life Cycle Controller escalation/RMA - Interrupt and alert outputs

Phase 4: Security Features - Secure wipe operations (DMEM, IMEM, internal state) - Error detection and classification - LOCKED state enforcement - LOAD_CHECKSUM computation

Phase 5: Algorithm Implementations - RSA-2048 using OpenSSL - Additional cryptographic algorithms as needed - Algorithm timing and instruction count estimation

Phase 6: Integration and Validation - System-level testbench development - Firmware driver validation - Performance characterization - Documentation completion

8.4 Key Differentiators

The OTBN SystemC TLM model differentiates itself through:

1. **No ISS Overhead:** Eliminates instruction fetch/decode/execute simulation overhead while maintaining functional correctness
2. **Library Integration:** Leverages established cryptographic libraries (OpenSSL) for algorithm implementations
3. **Scalable Algorithm Support:** Easily extends to new algorithms through base class interface
4. **Fast Software Development:** Enables early firmware development without waiting for RTL completion
5. **System-Level Focus:** Optimizes for virtual platform integration rather than RTL verification

8.5 Next Steps

Following approval of this high-level design specification:

1. **Detailed Design Document:** Generate comprehensive detailed design with class diagrams, sequence diagrams, and API specifications
2. **Test Plan Development:** Create SystemC testbench plans covering all functional scenarios
3. **Implementation:** Begin phased implementation per roadmap above
4. **Firmware Integration:** Coordinate with software teams for driver development and validation

8.6 Document Maintenance

This specification is a living document that will be updated as:

- Additional algorithms are identified for implementation
- Integration requirements evolve
- Firmware validation reveals functional gaps
- Performance optimization opportunities emerge

All updates will be versioned and reviewed through the standard design review process.

End of High-Level Design Specification